Robust Lane Detection for Challenging Scenarios: Adaptive ROI Extraction with Unified Local–Global Embedding Alignment

Jae Yun Lee*, Seonho Lee*, Sang Hoon Han*, Geun Ha Kim*, Student Member, IEEE, Jae Hyeon Park, Gyoomin Lee, and Sung In Cho, Member, IEEE

***Abstract*—Lane detection plays a pivotal role in autonomous driving and advanced driver assistance systems (ADAS). However, maintaining high accuracy under complex real-world conditions such as overpass shadows, sun glare, dense traffic, low light/nighttime scenes, and sharply curved roads remains a significant challenge. Among existing approaches, anchor-based methods that utilize region of interest (ROI) features to detect lanes currently yield the best detection performance, but they often lose local contextual information due to rigid, uniform ROI sampling along anchors. This sampling—performed at fixed, evenly spaced intervals—fails to concentrate feature extraction on highly curved segments where representational capacity is most needed. Consequently, fine-grained information of curves is under-sampled, diminishing expressiveness and detection accuracy in these difficult conditions. Additionally, in terms of existing attention mechanism combining the local context and the global context, treating the ROI feature as the sole query while using the global feature map as key and value exacerbates embedding misalignment: ROI features encode fine-grained local textures, whereas global key/value features capture holistic scene semantics, and their disparate information distributions prevent proper alignment, making lane detection under tough conditions even harder. This paper proposes an Adaptive Anchor Aggregation and Unified Embedding Alignment Network (A3-Net), which leverages the Adaptive Anchor Aggregation Module (AAAM) to adaptively pool crucial ROI features based on lane shape, and the Unified Embedding Alignment Module (UEAM) to align local and global contextual embeddings, thereby boosting robustness in challenging environments. The proposed method achieves state-of-the-art performance on the CULane benchmark, with an overall F1 score of 81.44% (+0.32%) and superior accuracy across all categories— normal lanes (+0.19%), crowded scenes (+0.03%), dazzle (+0.48%), shadow (+0.62%), no line (+1.3%), nighttime (+0.36%), and curved roads (+2.54%).**

***Index Terms*—Anchor-based, Autonomous driving, Challenge scenarios, Deep learning**

I. Introduction

WITH the recent breakthroughs in deep learning technologies, lane detection has emerged as a core component of autonomous driving systems and advanced driver assistance systems (ADAS) [1], [2], [3], [4], [5]. It plays a vital role in providing essential information for road structure understanding, path planning, collision avoidance, and overall driving safety. The process of detecting lanes from images captured by a front-facing camera is particularly critical for ensuring rapid response to unexpected situations and enhancing driving stability. By accurately identifying lane markings, autonomous vehicles can operate within a safer and more structured driving environment.

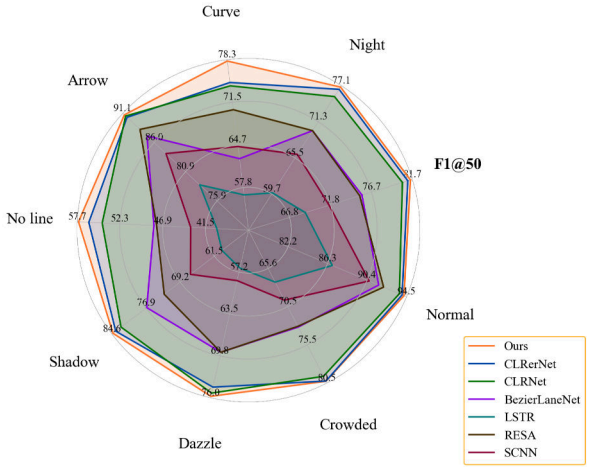


Fig. 1. F1 scores on the CULane dataset [7] for state-of-the-art methods including segmentation-based, parameter-based, and anchor-based, along with our method, across various driving scenarios.

However, accurate lane detection remains a highly challenging task due to the wide variety of real-world road conditions. In particular, lane detection systems must cope with varying lighting conditions, shadows, sharply curved lanes, and occlusions. Existing lane detection methods have attempted to overcome these challenges through various strategies, which can be broadly categorized into segmentation-based [6], [7], [8], [19], parameter-based [9], [10], [11], [12], [19], and anchor-based approaches [13], [14], [15], [16], [17], [18], [19].

The segmentation-based methods classify each pixel in an image as either part of a lane or background. Subsequently, post-processing techniques such as clustering are applied to the detected lane pixels to reconstruct individual lane lines. This approach does not require prior assumptions about the shape or number of lanes, enabling flexible detection of diverse lane types and multiple lanes on the road. However, since the pixel-wise classification primarily focuses on local features of the lanes, its performance tends to degrade in challenging scenarios where global contextual understanding of the road structure is crucial, such as crowded or dazzle scenes. Moreover, this method typically involves computationally expensive post-processing algorithms, which limit its computational efficiency.

Parameter-based methods model lanes using polynomial representations. By predicting the coefficients of a predefined polynomial, these methods offer the advantage of fast infer-

*First Author, Second Author, Third Author and Fourth Author contributed equally to this work.

The authors are with the Department of AI software, Dongguk University, Seoul 04720, Republic of Korea (e-mail: stcaft@dongguk.edu; sunhozizi@dongguk.edu; leo4102@dongguk.edu; rlarmsgk2@hanyang.ac.kr; pjh0011@dongguk.edu; exceleaf@dongguk.edu; csi2267@dongguk.edu).

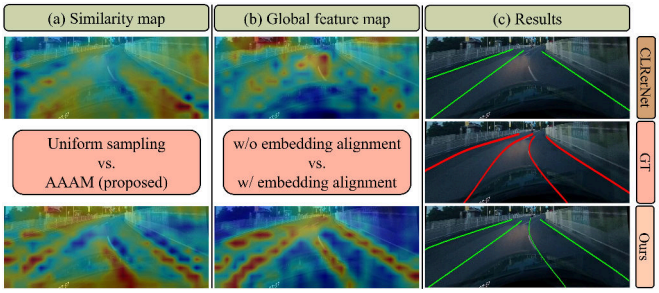


Fig. 2. Qualitative comparison between the existing SOTA method (CLRerNet) and our proposed method. (a) Similarity heat maps between ROI features and global feature maps. The top image shows CLRerNet using ROI features extracted by uniform sampling, while the bottom shows our method using adaptively selected ROI features through AAAM. (b) Visualization of global feature heat maps used in cross-attention with ROI features (queries). Our method shows enhanced focus around lane regions, indicating effective embedding alignment. (c) Lane detection results: predictions from CLRerNet (top), ground truth (middle), and our proposed method (bottom), demonstrating superior accuracy and lane continuity.

ence. Nevertheless, fixed-parameter polynomial models struggle to accurately capture complex lane geometries. As a result, they show low detection accuracy in scenarios such as curved lanes, where the use of high-order polynomials while intended to enhance representational flexibility, increases sensitivity to prediction errors, making it difficult to detect lanes reliably. Furthermore, employing high-order polynomials to improve representational flexibility increases detection accuracy variation making it difficult to detect lanes reliably.

Anchor-based methods initially extract global features using a pretrained image classification backbone. Then, they define straight-line anchors, each represented as a set of two-dimensional points within the image, to approximate the structure of candidate lanes. The predefined anchors are used to pool the feature values (or pixels) corresponding to each anchor region for a given image, which are then uniformly sampled to obtain region of interest (ROI) features. These ROI features are further elaborated through attention mechanisms with global features, resulting in ROI representations enriched with global context. By incorporating global context, the attention-refined ROI features enable robust lane detection even under occlusion. This anchor-based approach can be regarded as a hybrid strategy that combines the strengths of both segmentation-based and parameter-based methods. By adjusting the points that constitute each anchor, anchor-based methods allow for more fine-grained lane representation compared to parameter-based methods, which rely on a fixed number of polynomial coefficients. In addition, unlike segmentation-based methods that operate on a pixel-wise level, anchor-based approaches leverage the local feature extracted from the linear anchor, enabling more balanced detection that is not overly biased toward local regions. Consequently, recent anchor-based methods [17], [18] have demonstrated superior performance in challenging scenarios compared to traditional segmentation-based and parameter-based approaches. Nevertheless, existing anchor-based methods [13], [14], [15], [16], [17], [18], [19] still exhibit limitations in scenarios that require local and global contextual understanding, such as curves and dazzle environments, respectively. These methods face three primary challenges: First, they struggle to detect lanes in localized regions where lane shapes undergo significant variations, such as when lanes are highly curved or partially missing. This limitation stems from the uniform sampling of features along anchors, which are defined as straight lines. While this uniform-sampling strategy performs robustly under standard road conditions, it fails to adequately capture lane curvature in scenarios with highly tilted lanes, since it relies excessively on the anchors' fixed, elongated structural prior. Second, current anchor-based approaches compress ROI features into scalar representations during the attention process with global features. This scalar-based compression severely limits the representational capacity of ROI features, making it difficult to preserve fine-grained lane details. Additionally, this approach is insufficient for fully incorporating the global contextual information embedded in the global features. Third, due to the inherently different characteristics of ROI features (which represent local regions) and global features (which represent broader spatial context), there exists an embedding mismatch between the two. This discrepancy hinders effective attention process, ultimately degrading performance in scenarios that require rich semantic understanding for accurate lane detection. Specifically, existing attention mechanisms adopt a unidirectional cross-attention strategy, where ROI features are used as queries, and global features are used as keys and values. However, when the global features are only used to enhance the ROI features in a unidirectional manner, there is a risk of losing critical local patterns — such as lane curvature, length, and starting points — originally contained in the ROI features. This observation highlights the necessity of aligning global and ROI features through a more balanced interaction, where the global features are also enhanced based on the information contained in the ROI features.

In this paper, we propose two novel modules to address the aforementioned limitations and achieve superior performance compared to existing state-of-the-art methods, as illustrated in Fig. 1. First, we introduce an Adaptive Anchor Aggregation Module (AAAM), which enables the model to autonomously extract informative regions required for lane detection, rather than relying on uniformly sampled features along pre-defined linear anchors. With AAAM, ROI features can be extracted from regions containing richer lane-related information, allowing for more precise lane representation. Second, we propose a Unified Embedding Alignment Module (UEAM), which replaces the conventional unidirectional cross-attention mechanism with a bidirectional cross-attention mechanism. In UEAM, both the global feature map and ROI features are utilized as queries, enabling effective feature alignment between local and global features. This module encourages the global features, which capture scene-level context, to actively reference the ROI features containing local details. Consequently, the mutual enhancement of local and global features allows the model to understand not only the overall road layout but also the fine-grained structure of lane markings, contributing to accurate lane shape reconstruction. Fig. 2 visually illustrates the limitations of existing methods and the effectiveness of the proposed modules. The first row

of Fig. 2(a) shows a similarity heatmap between uniformly sampled ROI features and the global feature map, where high similarity is observed even in non-lane regions, highlighting a key limitation. In contrast, the heatmap obtained using selectively extracted ROI features through the AAAM module demonstrates a clearer distinction between lane and non-lane areas in terms of similarity. Fig. 2(b) visualizes the global feature map during the cross-attention process where the ROI features serve as the query. It can be observed that the global features, enhanced through the global pathway of the UEAM module, exhibit higher activation values for lane regions. This indicates that the global feature map becomes better aligned with the ROI features, allowing the cross-attention mechanism to compute higher similarity scores for lane regions compared to previous methods. Consequently, as shown in Fig. 2(c), the UEAM module enables the global features to effectively capture the local characteristics of the ROI features, leading to more precise lane detection than existing state-of-the-art approaches.

Through these contributions, the proposed method exhibits superior performance in challenging scenarios involving occlusions or extreme curvature, while also maintaining competitive performance in standard driving environments, outperforming existing state-of-the-art methods. The main contributions of this paper are as follows:

1) We propose an Adaptive Anchor Aggregation Module (AAAM) that adaptively extracts each anchor's features based on the lane and road conditions within the image. This approach overcomes the limitations of uniform sampling and achieves superior lane detection performance in scenarios that require detailed local context.
2) We introduce a Unified Embedding Alignment Module (UEAM) that maps both the global features and ROI features into a shared embedding space. By applying attention between the two, the module precisely measures similarity and produces ROI features enriched with global context.
3) By integrating AAAM and UEAM, our framework simultaneously performs mutual enhancement of local and global features, achieving state-of-the-art performance on challenging scenarios in the CULane [7] and CurveLanes [20] benchmark datasets.

II. RELATED WORK

A. Segmentation-based Methods

The segmentation-based approach [6], [7], [8], [19] formulates the lane detection task as a semantic segmentation problem. In this approach, every pixel in the input image is classified as either a lane or background, and then the classified pixels are grouped into individual lane instances through a post-processing step, such as clustering, to represent multiple lanes.

A representative example is SCNN [7], which is one of the early approaches for lane detection. While conventional convolutional neural networks (CNNs) are effective at extracting local features, they have limitations in capturing the relationships between distant pixels within an image. To address this problem, SCNN divides feature maps into slices (either rows or columns of the image) and sequentially performs convolution operations on each slice. Through this sequential message-passing mechanism within the same layer, SCNN enables information to be propagated across distant pixels. However, SCNN adopts a sequential message-passing scheme in which information cannot be transferred to the next slice until the current slice has been updated. This sequential processing makes it difficult to perform parallel computation, resulting in slow inference speed.

To overcome these limitations, RESA [8] proposes a recurrent feature-shifting mechanism that enables parallel updates across all locations simultaneously. Instead of sequentially propagating messages to adjacent slices as in SCNN, RESA delivers information in multiple directions (i.e., up, down, left, and right) in a parallel manner. By applying different strides, RESA can also effectively aggregate information from distant regions beyond just neighboring slices. Moreover, RESA enhances lane feature representation by applying a feature-shifting mechanism, which repeatedly shifts the feature maps vertically and horizontally to capture fine-grained spatial relationships between pixels across rows and columns. Through this design, RESA achieves faster inference speed and addresses the lack of global context in previous methods.

B. Parameter-based Methods

Parameter-based lane detection methods [9], [10], [11], [12], [19] provide fast inference speed by modeling lanes through the prediction of polynomial coefficients, curve control points, or principal points that are critical for lane detection, instead of directly predicting all points along the lane. Conventional parameter-based lane detection methods generally consist of two stages: feature extraction and post-processing. In the first stage, feature extraction aims to represent lanes in a parametric form by predicting polynomial coefficients, curve control points, or principal points from the input image. In the subsequent post-processing stage, the extracted lane features are further processed, such as through clustering, to distinguish individual lanes and obtain the parameters necessary for lane detection, including polynomial coefficients or curve control points. A representative approach is polynomial-based methods [10], which model lanes by predicting polynomial coefficients. While this approach is useful for capturing the curvature of lanes, it has a drawback in that small changes in the coefficients can lead to significant variations in the lane shape. To address this limitation, BezierLaneNet [11] models lanes using Bezier curves instead of polynomial curves. Bezier curves are more stable than polynomial curves, as changes in control points have less influence on the overall shape of the curve, allowing the model to learn lane shapes more robustly and precisely. In addition, BezierLaneNet introduces a feature flip fusion technique that leverages the bilateral symmetry of roads by fusing features obtained from horizontally flipped images, thereby enhancing the robustness of lane predictions. As a result, BezierLaneNet achieved higher accuracy compared to conventional polynomial-based methods. Another parameter-based approach, Point-2-Lane [12], reformulates the

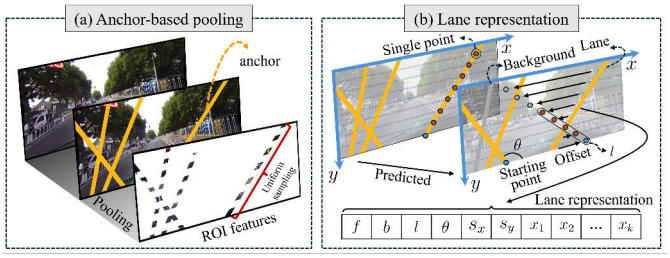


Fig. 3. Overview of the anchor-based pooling process and lane representation. (a) demonstrates the pooling process, where ROI features are pooled from the image using predefined anchors based on a straight-shaped lane prior. (b) visualizes the lane representation, which consists of multiple parameters: f and b represent the foreground and background probabilities, l denotes the length of the predicted lane, θ defines the angle between the lane and the positive x-axis, s_x and s_y indicate the x- and y-coordinates of the lane's starting point, and x_k represents the k-th x-coordinate offset.

lane detection task as the problem of detecting principal points (PP) for each lane. This method first identifies the start and end points of lanes from the ground truth (GT) and then selects PPs that play an important role in representing the curve to re-partition the lane. This enables more flexible lane detection by utilizing a larger number of PPs for complex curved lanes and fewer PPs for simple straight lanes. However, this two-stage approach mainly focuses on extracting local lane features within the image, which limits its ability to effectively capture the global context of the scene. To overcome this limitation, LSTR [9] proposes an end-to-end approach based on Transformer [21]. LSTR removes the conventional post-processing step and directly predicts a limited number of curve parameters from the input image. In particular, the self-attention mechanism of the Transformer effectively captures non-local interactions between distant pixels, enabling the model to incorporate global context. With this architecture, LSTR demonstrates more robust lane detection performance compared to previous parameter-based methods. However, parameter-based methods still have limitations in capturing fine-grained lane details, as they represent a large number of points along the lane using only a small set of parameters.

C. Anchor-based Methods

Segmentation-based methods classify every pixel in the image, effectively capturing local information and allowing flexible lane shape representation. In contrast, parameter-based methods model lanes using fixed polynomial coefficients or control points, which focus more on capturing global semantic context than segmentation-based methods but have limited flexibility in lane representation. As a compromise between these two approaches, anchor-based methods [13], [14], [15], [16], [17], [18], [19] have emerged. As shown in Fig. 3(a), this approach utilizes predefined anchors in regions where lanes are likely to exist to extract regions of interest (ROIs), enabling more effective utilization of global context compared to segmentation-based methods. In addition, as shown in Fig. 3(b), anchor-based methods can represent lanes more flexibly than parameter-based methods by using a larger number of points to describe each anchor. As a result, recent anchor-based methods have achieved superior lane detection performance compared to previous approaches. Anchor-based methods can be broadly categorized into row anchor-based methods [13], [14] and line anchor-based methods [15], [16], [17], [18].

Row anchor-based methods [13], [14] divide the image into fixed horizontal rows and predict the x-coordinate in each row where a lane is likely to exist. In this approach, a lane instance is represented by a set of x-coordinates corresponding to each row in the image, and these points are connected to form the complete lane. Ultra-Fast Lane Detection (UFLD) [13] was the first to reformulate the lane detection problem as an anchor-driven ordinal classification task instead of the conventional pixel-wise segmentation approach. By representing lanes with row-wise coordinates for only a part of the image rather than processing all pixels in the entire image, UFLD significantly reduces computational costs. UFLD [13] adopts ResNet-18 [22], a lightweight backbone designed for image classification on the ImageNet dataset [23], to minimize computational overhead; however, it suffers from relatively lower overall lane detection performance.

Line anchor-based methods [15], [16], [17], [18] utilize predefined line-shaped anchors, where features are pooled and then sampled from the corresponding locations in the image or feature map to obtain the candidate ROI features for lane detection. Then, as shown in Fig. 3(b), the model predicts the offset, which represents the distance between the points on the ground truth (GT) lane and the corresponding points on the line-shaped anchor, to refine the lane position based on each anchor. This approach allows the line-shaped anchors to serve as a reference that represents the overall structure of the lane, enabling the model to effectively capture the global shape of the lane. Line-CNN [15] first introduces the line anchor-based method, and LaneATT [16] extracts local features corresponding to each predefined anchor from the feature map. However, instead of simply performing sampling, it introduces a novel anchor-based attention mechanism. This method applies attention between the local ROI features of each anchor and the global features aggregated from other anchors. By leveraging this mechanism, the model is designed to accurately detect lane positions even in challenging scenarios, such as when lanes are occluded or when lane markings are missing, by effectively utilizing the global context. In addition, CLRNet [17] detects lanes by applying an attention mechanism between multi-scale features obtained from a feature pyramid network [24] and the RoI features extracted through anchors. Furthermore, CLRNet introduces a new loss function, LineIoU [17], which effectively computes the IoU [25] between the predicted lanes and the ground truth lanes, providing correct optimization guidance during training. CLRerNet [18] further improves CLRNet by modifying LineIoU to enable accurate IoU calculation even for lanes with large curvature. Both CLRNet and CLRerNet have inspired our method, and we describe them in more detail in Section II-E for a better understanding of our approach.

D. Dual Vision Transformer

Dual Vision Transformer (Dual ViT) [26] proposes a novel "dual block" to effectively handle high-resolution inputs while

significantly reducing computational cost and accurately capturing both global semantic information and fine-grained details. The dual block consists of two main pathways: the pixel pathway and the semantic pathway. The pixel pathway refines fine-grained features by incorporating local context for each pixel of the input feature map. This pathway enhances detailed representations in high-resolution inputs by effectively aggregating surrounding information for each pixel. In contrast, the semantic pathway extracts semantic features that capture the global information of the entire image. These features serve as abstract representations summarizing the overall meaning of the image and are used as important prior information to guide the pixel pathway in learning fine-grained details. The outputs of the two pathways are fused in the merge block, which is then fed into the multi-head self-attention layer. As a result, the computational complexity of the self-attention operation is reduced from $O(n^2d)$ to $O(n \cdot md + m^2d)$, enabling a significant reduction in computation even for high-resolution inputs. Inspired by this concept, our approach adopts a similar dual block structure that leverages ROI features and global features to extract both fine-grained lane details and global context effectively.

E. CLRNet and CLRerNet

To aid in the understanding of our proposed method, we explain the anchor-based pooling technique and lane representation utilized in CLRNet [17] and CLRerNet [18].

Fig. 3(a) visually illustrates the process in which pixels or features corresponding to the thin and elongated anchors are pooled, followed by uniform sampling. As described earlier, both methods sample ROI features at uniform intervals along the line-shaped anchors from the global feature map. However, this uniform sampling strategy has a limitation in that it degrades the detection performance for complex lanes such as curved lanes. As shown in Fig. 3(b), the lane representation describes a lane in an image by predicting the existence of the lane, its starting point, length, angle, and 72 points along the lane. This representation regularizes the lane shape, allowing the model to effectively reduce the search space and simplify the lane detection problem.

The lane representation consists of the following: (1) f and b indicating the probabilities that the anchor corresponds to a lane (foreground) or not (background), respectively. (2) l, the length of the predicted lane, (3) θ, the angle that the anchor forms with the positive direction of the x-axis, (4) s_x and s_y, the x- and y-coordinates representing the starting point of the anchor, and (5) x_k, the k-th x-coordinate offset representing how far the predicted lane is from the anchor in the x-axis direction. This representation depicts lanes and aids in model training by corresponding to the y-axis coordinates with a predefined number of offsets.

After extracting ROI features using the above lane representation and anchors, both CLRNet and CLRerNet commonly apply an attention mechanism where the global feature map is used as keys and values, and the ROI feature is compressed into a single scalar value to serve as the query. Through this process, the ROI feature is enhanced by incorporating the global context contained in the global feature map. However, as shown in Fig. 2(b), due to the characteristic difference between the ROI feature, which contains local context information, and the global feature, which contains global context information, the similarity-based attention mechanism may have limited effectiveness in exchanging features between them, leaving room for improvement.

After the attention operation with the global feature, the enhanced ROI feature is transformed into a lane representation, which is then used to calculate the Line Intersection over Union (LineIoU) [17] loss with the ground truth. Through this training process, the model addresses the lane detection task.

CLRerNet is an extended version of CLRNet that modifies the LineIoU calculation method of CLRNet. In the original CLRNet, the LineIoU computes the IoU of lanes using a fixed lane width along the horizontal direction, which has limitations in accurately reflecting the shape of largely tilted lanes. To address this issue, the improved LaneIoU dynamically adjusts the lane width according to the slope of the lane in each row, enhancing the precision of IoU calculation. However, both CLRNet and CLRerNet rely on fixed anchor features and suffer from the imbalance between the ROI features and the global feature map, making it difficult to achieve high lane detection performance in challenging scenarios.

III. PROPOSED METHOD

A. Overall Method

Fig. 4 illustrates the overall process of the proposed Adaptive anchor Aggregation and unified embedding Alignment Network (A3-Net). As shown in Fig. 4, the input image is resized to 320×800 and then passed through the backbone network. The feature map extracted from the backbone undergoes upsampling and convolution operations within the FPN [24] to produce a global feature map with multiple resolutions.

Subsequently, the global feature map ($\boldsymbol{\mathcal{Z}} \in \mathbb{R}^{H\times W\times C}$) serves as the input to the head and is used to extract ROI features ($\boldsymbol{\mathcal{X}} \in \mathbb{R}^{N_a\times C\times S}$) based on the pre-defined anchors. $N_a(192)$ denotes the number of anchors, $C(64)$ is the size of the feature channels, and $S(72)$ represents the number of each anchor's points. The head is composed of the proposed Adaptive Anchor Aggregation Module (AAAM) and Unified Embedding Alignment Module (UEAM). Specifically, within each head, AAAM follows the pooling process illustrated in Fig. 3. It pools the feature values along the anchors from the global feature map ($\boldsymbol{\mathcal{Z}}$) to obtain ROI features ($\boldsymbol{\mathcal{X}}$). To prevent mixing of features between different anchors, individual convolutions are applied separately for each anchor. Subsequently, a linear layer is employed to allow the model to autonomously extract adaptively selected ROI features ($\widehat{\boldsymbol{\mathcal{X}}} \in \mathbb{R}^{N_a\times C\times D}$), where D (9) indicates the number of adaptively selected features per anchor.

Following AAAM, the Unified Embedding Alignment Module (UEAM) takes as input the flattened global feature ($\boldsymbol{z} \in \mathbb{R}^{HW\times C}$) and each sliced feature ($\hat{\boldsymbol{x}} \in \mathbb{R}^{N_a\times C}$) along the D-axis from the adaptively selected ROI feature ($\widehat{\boldsymbol{\mathcal{X}}}$). These features are separately passed through a global pathway and a ROI pathway, producing a flattened global feature map

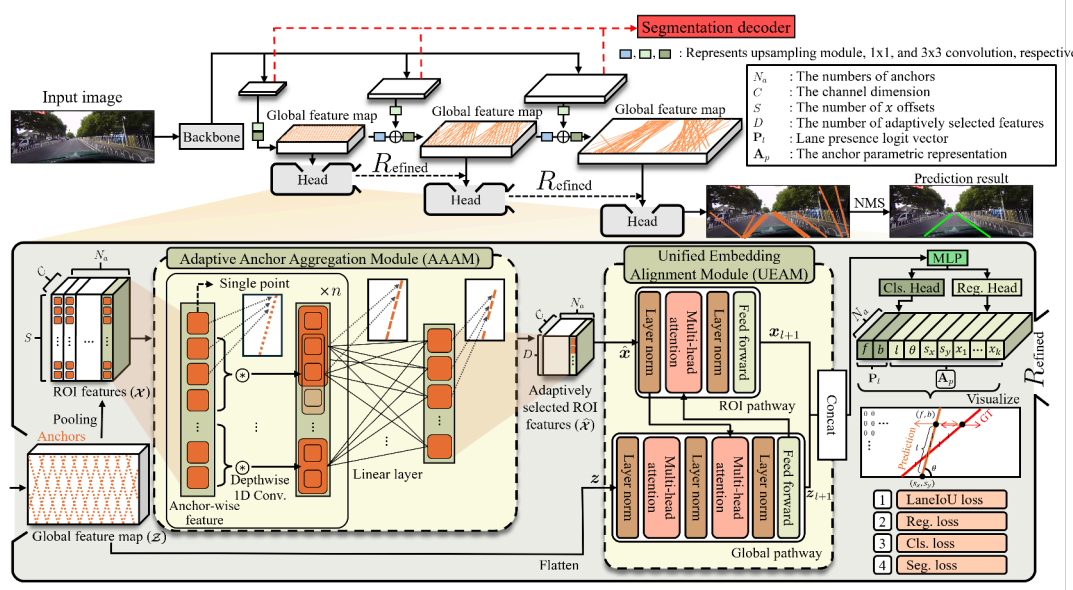


Fig. 4. Overview of the proposed model: The proposed model utilizes an FPN with three feature levels, each processed by a separate head. Each head consists of the AAAM and the UEAM. The outputs of each head include $\mathbf{P}_l$ and $\mathbf{A}_p$, which are optimized using LaneIoU [18], regression, classification, and segmentation losses.

($\boldsymbol{z}_{l+1} \in \mathbb{R}^{HW \times C}$) and sliced ROI feature ($\boldsymbol{x}_{l+1} \in \mathbb{R}^{N_a \times C}$), respectively.

The two outputs are then concatenated and passed through a multi-layer perceptron (MLP) consisting of three linear layers and two ReLU [27] activations, resulting in a unified feature that integrates both local and global contexts. The unified feature is subsequently passed to a classification head and a regression head, each consisting of three linear layers with two ReLU activation functions between them. As shown in Fig. 4, the classification head outputs a logit vector ($\mathbf{P}_l \in \mathbb{R}^{N_a \times 2}$), representing the probabilities of each anchor corresponding to either a lane or the background. Meanwhile, the regression head outputs a parametric representation ($\mathbf{A}_p \in \mathbb{R}^{N_a \times 76}$), which encodes the length, angle, starting point, and 72 x-coordinates of the predicted lane. This lane representation matches the formulation described in Fig. 3(b). The operations of the head are applied three times, each based on a different resolution of the global feature maps extracted by the FPN. Consequently, the lane representation vectors, which are the output from the head, consist of classification logits $\mathbf{P}_l$ and parametric representations $\mathbf{A}_p$. The parametric representations $\mathbf{A}_p$ are further utilized in the next stage to refine the anchors, ensuring they are better aligned with the actual lane shapes.

B. Backbone, Feature Pyramid Network, and Head

To obtain a multi-level feature map from the input image capturing both rich semantic context and fine-grained localization details, we use the feature pyramid network (FPN) that uses the residual network (ResNet) [22] (or deep layer aggregation (DLA) [28]) as the pre-trained backbone. FPN consists of three levels, where the feature resolution ($\in \mathbb{R}^{H \times W \times C}$) progressively increases at each level by a factor of two in both height and width (i.e., $2\times$ upsampling per level starting from $H(10)$, $W(25)$ and $C(64)$), thereby enabling the network to capture progressively finer spatial details. Global feature maps which are extracted from each level of FPN have different spatial resolutions and are sequentially utilized as inputs to the refinement heads.

C. Adaptive Anchor Aggregation Module (AAAM)

The Adaptive Anchor Aggregation Module (AAAM) extracts adaptively selected ROI features based on the positional importance of features within each anchor, rather than uniformly sampling the ROI spatially. Specifically, it evaluates the importance of each position in the anchor-wise feature ($\in \mathbb{R}^{C \times S}$). As illustrated in Fig. 4, the feature dimension is reduced from S (72) to D (9), resulting in adaptively selected ROI features ($\boldsymbol{\mathcal{X}} \in \mathbb{R}^{N_a \times C \times D}$). Each of the D outputs in the adaptively selected ROI features ($\widehat{\boldsymbol{\mathcal{X}}}$) captures local contextual characteristics of the lane structure.

To extract the adaptively selected ROI features ($\widehat{\boldsymbol{\mathcal{X}}}$), AAAM employs depthwise one-dimensional convolution (dwConv1D) [29] n times, denoted as $\circledast$ along each anchor, ensuring that anchor-specific information is preserved without mixing. This process is followed by a linear layer to extract features critical for lane reconstruction, as shown in Fig. 4. The overall operation of AAAM can be expressed as:

$$\widehat{\boldsymbol{\mathcal{X}}} = \text{Linear}(\text{dwConv1D}(\text{dwConv1D}(\boldsymbol{\mathcal{X}}))). \quad (1)$$

Through AAAM, we reconstruct features by focusing on locations with higher information content among the S points of each anchor, thereby achieving a more appropriate representation for the lane structure. The dramatic effect of the proposed AAAM in feature aggregation is visually demonstrated through the attention maps shown in Fig. 2(a).

The output $\widehat{\boldsymbol{\mathcal{X}}}$ from AAAM is then used as the input to the Unified Embedding Alignment Module (UEAM). Note that, in UEAM, attention is not applied to the entire adaptively

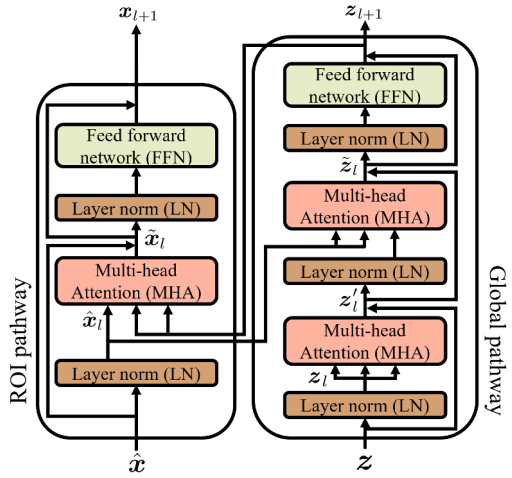


Fig. 5. Overall structure of the UEAM consisting of ROI and global pathways.

selected ROI feature ($\widehat{\boldsymbol{\mathcal{X}}}$) at once. Instead, attention operations in UEAM are sequentially performed between each slice along the D-axis ($\hat{\boldsymbol{x}} \in \mathbb{R}^{N_a \times C}$) and the flattened global feature map ($\boldsymbol{z} \in \mathbb{R}^{HW \times C}$).

D. Unified Embedding Alignment Module (UEAM)

When calculating the similarity between the ROI feature and the global feature map, the ROI feature serves as a query that can identify regions likely to contain lanes on the global feature map, as used in the existing methods [17], [18]. However, the ROI feature mainly contains local, low-level information, whereas the global feature map captures global, high-level semantics. Because existing methods employ only a unidirectional query from the ROI feature, a discrepancy in feature embeddings arises between the ROI features obtained from the previous head and the global features extracted in the current head. This discrepancy becomes even more pronounced due to the use of feature maps with different resolutions from the feature pyramid network (FPN), making it difficult to accurately measure similarity based on attention. As a result, as shown in Fig. 2(b), the attention is not sufficiently focused on the lane regions, leading to insufficient incorporation of global context information.

To address this issue, we propose the Unified Embedding Alignment Module (UEAM). Inspired by the dual block architecture of the Dual Vision Transformer [26], UEAM consists of two pathways. As illustrated in Fig. 5, the left branch represents the ROI pathway, and the right branch represents the global pathway. The ROI pathway takes as input the sliced features ($\hat{\boldsymbol{x}} \in \mathbb{R}^{N_a \times C}$) obtained by slicing the output of AAAM ($\hat{\mathcal{X}} \in \mathbb{R}^{N_a \times C \times D}$) along the D-axis, whereas the global pathway takes as input the flattened global feature ($\boldsymbol{z} \in \mathbb{R}^{HW \times C}$) derived from the global feature map ($\boldsymbol{\mathcal{Z}} \in \mathbb{R}^{H \times W \times C}$). Both pathways utilize layer normalization (LN), multi-head attention (MHA), and a feed-forward network (FFN). As shown in Fig. 2(b), the global and ROI pathways are aligned within a shared embedding space, enabling effective cross-attention between local and global features. The initial operations of the global pathway are conducted as follows:

$$\begin{aligned} \boldsymbol{z}_l &= \mathrm{LN}(\boldsymbol{z}), \\ \boldsymbol{z}'_l &= \mathrm{MHA}(\boldsymbol{z}_l, \boldsymbol{z}_l, \boldsymbol{z}_l) + \boldsymbol{z}. \end{aligned} \tag{2}$$

Specifically, in the global pathway, the input global feature map ($\mathcal{Z}$) from the head is flattened along its spatial dimensions to obtain the global feature ($\boldsymbol{z}$). Here, C denotes the number of channels, while H and W represent the height and width of the global feature map, respectively. The flattened global feature $\boldsymbol{z}$ is first passed through a layer normalization (LN) to obtain $\boldsymbol{z}_l$, which is then used as the query, key, and value in the multi-head attention (MHA) mechanism. After processing through the MHA, the output ($\boldsymbol{z}'_l \in \mathbb{R}^{HW \times C}$) is obtained. The intermediate operations in the global pathway are conducted as follows:

$$\begin{aligned} \hat{\boldsymbol{x}}_l &= \mathrm{LN}(\widehat{\boldsymbol{x}}), \\ \tilde{\boldsymbol{z}}_l &= \mathrm{MHA}(\mathrm{LN}(\boldsymbol{z}'_l), \hat{\boldsymbol{x}}_l, \hat{\boldsymbol{x}}_l) + \boldsymbol{z}'_l, \\ \boldsymbol{z}_{l+1} &= \mathrm{FFN}(\mathrm{LN}(\tilde{\boldsymbol{z}}_l)) + \tilde{\boldsymbol{z}}_l. \end{aligned} \tag{3}$$

The layer-normalized ROI feature ($\hat{\boldsymbol{x}}_l \in \mathbb{R}^{N_a \times C}$) at the initial stage of the ROI pathway serves as the key and value, while $\boldsymbol{z}'_l$, obtained after MHA in the global pathway, serves as the query. Through this process, the final output of the global pathway becomes the enhanced global feature map ($\boldsymbol{z}_{l+1} \in \mathbb{R}^{HW \times C}$), obtained via cross-attention with the layer-normalized ROI feature ($\hat{\boldsymbol{x}}_l$). Subsequently, in the ROI pathway, the ROI feature ($\hat{\boldsymbol{x}} \in \mathbb{R}^{N_a \times C}$) is used as input. The ROI feature ($\hat{\boldsymbol{x}}$) first undergoes layer normalization and then performs cross-attention with the output ($\boldsymbol{z}_{l+1}$) from the global pathway. At this point, the layer-normalized ROI feature ($\hat{\boldsymbol{x}}_l$) is used as the query, and the global feature ($\boldsymbol{z}_{l+1}$) is used as the key and value. The final output of the ROI pathway is the enhanced ROI feature ($\boldsymbol{x}_{l+1} \in \mathbb{R}^{N_a \times C}$), obtained after the sequence of cross-attention, layer normalization, and feed-forward network operations. The detailed operations can be described by the following equations:

$$\begin{aligned} \tilde{\boldsymbol{x}}_l &= \mathrm{MHA}(\hat{\boldsymbol{x}}_l, \boldsymbol{z}_{l+1}, \boldsymbol{z}_{l+1}) + \hat{\boldsymbol{x}}_l, \\ \boldsymbol{x}_{l+1} &= \mathrm{FFN}(\mathrm{LN}(\tilde{\boldsymbol{x}}_l)) + \tilde{\boldsymbol{x}}_l. \end{aligned} \tag{4}$$

The feature ($\boldsymbol{x}_{l+1}$) obtained after cross-attention with the global feature ($\boldsymbol{z}_{l+1}$) is concatenated with the global feature ($\boldsymbol{z}_{l+1}$) along the batch dimension, resulting in a feature of size ($\in \mathbb{R}^{(N_a+HW) \times C}$), as shown in Fig. 4. This concatenated feature is then fed into an MLP composed of three linear layers and two activations. The MLP processes the combined features, considering both the global feature and the ROI feature, and outputs a feature of size ($\in \mathbb{R}^{N_a \times C}$), which serves as the input to the classification head and regression head illustrated in Fig. 4. This entire process is repeated D times, considering that the output ($\boldsymbol{x}_{l+1}$) of UEAM, originally corresponds to an input feature ($\hat{\boldsymbol{x}} \in \mathbb{R}^{N_a \times C}$) obtained by slicing the output ($\widehat{\boldsymbol{\mathcal{X}}}$) of AAAM along the D-axis.

E. Classification and Regression for Lane Representation

As described earlier, the ROI feature ($\boldsymbol{x}_{l+1} \in \mathbb{R}^{N_a \times C}$) and the global feature ($\boldsymbol{z}_{l+1} \in \mathbb{R}^{HW \times C}$) produced by the UEAM module are concatenated ($\in \mathbb{R}^{(N_a+HW) \times C}$) and then passed

as input to the multi-layer perceptron (MLP) ($\in \mathbb{R}^{N_a \times C}$) shown in Fig. 4. This process is repeated D times, and as a result, we obtain the final feature ($\in \mathbb{R}^{N_a \times CD}$) by concatenating along the channel dimension that serves as the input to the classification and regression heads. The resulting feature ($\in \mathbb{R}^{N_a \times CD}$) is further fed into the classification head and regression head, where each head processes N_a anchors. The classification head, composed of two linear layers (each followed by a ReLU activation) and a final linear layer, outputs a classification logit vector ($\mathbf{P}_l \in \mathbb{R}^{N_a \times 2}$), representing the foreground and background probabilities for each anchor. The regression head, composed of two linear layers (each followed by a ReLU activation) and a final linear layer, outputs a parametric representation vector ($\mathbf{A}_p \in \mathbb{R}^{N_a \times 76}$). Finally, during inference, the outputs of the final head, $\mathbf{P}_l$ and $\mathbf{A}_p$ are passed through a non-maximum suppression (NMS) step [16] to allow the model to detect lane lines from the input image.

F. Training Loss

The proposed method employs four types of loss functions: Lane Intersection over Union (IoU) loss [18], regression loss, classification loss, and segmentation loss. Before calculating each loss, we first perform dynamic matching between the total N_a anchors and a relatively smaller number of ground truth (GT) lanes. To achieve this, we adopt the dynamic top-k assigner utilized in CLRerNet [18], dynamically selecting the top-k anchors with the highest IoU scores relative to each GT lane among the N_a anchors. For each GT lane, the selected anchors (predicted lanes) are paired with the corresponding GT lane parameters to compute the loss functions detailed below. Assuming there are G GT lanes within an image and that each GT lane is dynamically assigned k_m anchors, to understand the overall loss, we first define in Equation 5 the dynamically assigned anchors allocated to each GT lane:

$$\begin{aligned} K &= \sum_{m=1}^{G} k_m, \\ \mathcal{P} &= \{(m, r) | r \in \text{anchors assigned to GT } m\}. \end{aligned} \tag{5}$$

Here, G denotes the number of ground truth (GT) lanes, and K represents the total number of anchors dynamically assigned to all GT lanes. The set $\mathcal{P}$, consisting of k_m anchors, represents the collection of r predicted lanes assigned to the m-th GT lane. Through the use of the dynamic top-k assigner, we effectively select individual GT lanes and their corresponding predicted lanes, thereby reducing the computational burden during loss calculation.

First, we adopt the LaneIoU loss ($\mathcal{L}_{LaneIoU}$) utilized in previous work [18], calculating the Intersection over Union (IoU) between the ground truth lanes and the predicted lanes following the same procedure. LaneIoU improves upon the LineIoU proposed in CLRNet by dynamically assigning the lane width w based on the change in x-coordinates during the IoU calculation between the predicted and ground truth lanes. This dynamic assignment enables more accurate IoU computation for tilted lanes and complex curved shapes. For the i-th predicted x-coordinate ($\hat{x}_i$) and the corresponding y-coordinate (y_i) — obtained by uniformly dividing the input image along the y-axis — the assigned lane width (w) is defined as follows:

$$\begin{aligned} \Delta\hat{x}_i = \hat{x}_{i+1} - \hat{x}_{i-1}, \Delta y_i = y_{i+1} - y_{i-1}, \\ w_i = \frac{w_{\text{lane}}}{2} \frac{\sqrt{(\Delta\hat{x}_i)^2 + (\Delta y_i)^2}}{\Delta y_i}, \end{aligned} \tag{6}$$

where w_{lane} determines the strictness of the LaneIoU calculation. $\Delta\hat{x}_i$ and Δy_i represent the local changes of the i-th lane point along with each axis. Using the assigned lane width w_i in Equation 6, the intersection I_i and the union U_i for the i-th point are defined as follows, where x_i represents the i-th coordinate of the ground truth lane to which the predicted lane is assigned:

$$\begin{aligned} I_i &= \min(\hat{x}_i + w_i, x_i + w_i) - \max(\hat{x}_i - w_i, x_i - w_i), \\ U_i &= \max(\hat{x}_i + w_i, x_i + w_i) - \min(\hat{x}_i - w_i, x_i - w_i). \end{aligned} \tag{7}$$

Based on the definitions of I_i and U_i above, the LaneIoU loss is calculated as follows:

$$\mathcal{L}_{LaneIoU} = \frac{\sum_{i=0}^{H} I_i}{\sum_{i=0}^{H} U_i}, \tag{8}$$

where H denotes the number of divisions along the y-axis. Through this loss function, the model is guided to learn the overall structure of the ground truth lanes more effectively.

Second, the regression loss ($\mathcal{L}_{reg}$) uses the output of the regression head ($\mathbf{A}_p$) to calculate the differences between the predicted lane parameters and the ground truth (GT) lane parameters for four components: the length (l), angle (θ), starting x-point, and starting y-point. These differences are computed using the smooth-L1 loss [30]. Specifically, among the K predicted lanes dynamically assigned through the dynamic top-k assigner, we define the predicted lane representations, consisting of l, θ, s_x, and s_y, as $\hat{\boldsymbol{y}}_{reg} \in \mathbb{R}^{K \times 4}$, and the corresponding GT lane parameters as $\boldsymbol{y}_{reg} \in \mathbb{R}^{K \times 4}$. The regression loss is then calculated between $\hat{\boldsymbol{y}}_{reg}$ and $\boldsymbol{y}_{reg}$ as follows:

$$\mathcal{L}_{reg} = \frac{1}{K} \sum_{(m,r) \in \mathcal{P}} \sum_{j=1}^{4} \mathrm{L}_{smooth-L1}(\hat{y}_{reg_{m,r,j}}, y_{reg_{m,j}}), \tag{9}$$

where m and r are defined in Equation 5. Through this process, the predicted lane parameters that represent the overall shape of the lanes can be regressed to better match the ground truth (GT) lanes.

Third, the classification loss ($\mathcal{L}_{cls}$), is calculated using the output $\mathbf{P}_l$ from the classification head shown in Fig. 4. Unlike the previous loss functions, the classification loss calculation also incorporates predicted lanes that were not selected by the dynamic top-k assigner, by applying the focal loss [31]. Specifically, the logit vector $\mathbf{P}_l$ obtained from the classification head is passed through the softmax function $\phi(\cdot)$ to produce the probability vector $\hat{\boldsymbol{y}}$. For the a-th predicted lane, the probability vector $\hat{\boldsymbol{y}}$ consists of the probability of being a lane ($\hat{y}_{a,0}$) and the probability of being background ($\hat{y}_{a,1}$), satisfying $\hat{y}_{a,0} + \hat{y}_{a,1} = 1$. The corresponding label vector $\boldsymbol{y}_a$, used for calculating the focal loss, indicates the assigned label for the a-th prediction. For lanes selected by the dynamic top-k assigner, the labels are assigned as foreground lanes

TABLE I
F1@50 FOR EACH CATEGORY ON CULANE (EXCEPT FOR THE CROSS CATEGORY WHERE ONLY FALSE POSITIVES ARE SHOWN)

Method	Backbone	Total	Normal	Crowded	Dazzle	Shadow	No line	Arrow	Curve	Cross	Night
Segmentation-based											
SCNN [7]	VGG16	71.60	90.60	69.70	58.50	66.90	43.40	84.10	64.40	1990	66.10
RESA [8]	ResNet34	74.50	91.90	72.40	66.50	72.00	46.30	88.10	68.60	1896	69.80
	ResNet50	75.30	92.10	73.10	69.20	72.80	47.70	88.30	70.30	1503	69.90
UFLD [13]	ResNet18	68.40	87.70	66.00	58.40	62.80	40.20	81.00	57.90	1743	62.10
	ResNet34	72.30	90.70	70.20	59.50	69.30	44.40	85.70	69.50	2037	66.70
Lane2Seq (segmentation) [19]	ViT-Base	79.64	93.39	77.27	73.45	79.69	53.91	90.53	73.37	1129	74.96
Parameter-based											
LSTR [9]	-	68.72	86.78	67.36	56.63	59.82	40.10	78.66	56.64	1166	59.92
BezierLaneNet [11]	ResNet18	73.67	90.22	71.55	62.49	70.91	45.30	84.09	58.98	996	68.70
	ResNet34	75.57	91.59	73.20	69.20	76.74	48.05	87.16	62.45	**888**	69.90
Point2Lane [12]	ResNet18	77.47	92.24	77.68	72.05	61.30	54.13	88.28	65.02	1765	73.95
	ResNet34	79.45	93.93	79.93	73.53	71.71	55.81	89.35	68.63	1560	74.49
Lane2Seq (parameter) [19]	ViT-Base	79.27	93.11	77.43	73.25	79.46	53.74	90.02	72.44	1173	75.12
Anchor-based											
CLRNet [17]	ResNet18	79.58	93.30	78.33	73.71	79.66	53.14	90.25	71.56	1321	75.11
	ResNet34	79.73	93.49	78.06	74.57	79.92	54.04	90.59	72.77	1216	75.02
	ResNet101	80.13	93.85	78.78	72.49	82.33	54.50	89.79	75.57	1262	75.51
	DLA34	80.47	93.73	79.59	75.30	82.51	54.58	90.62	74.13	1155	75.37
CLRerNet [18]	ResNet34	80.76	93.93	79.51	73.88	83.16	55.55	**90.87**	74.45	1088	76.02
	ResNet101	80.91	93.91	80.03	72.98	82.92	55.73	90.53	73.83	1113	76.13
	DLA34	81.12	94.02	80.20	74.41	83.71	56.27	90.39	74.67	1161	76.53
Lane2Seq (anchor) [19]	ViT-Base	78.39	93.03	76.42	72.17	78.32	52.89	89.67	72.67	1319	73.98
A3-Net (ours)	ResNet18	80.66	93.92	79.34	73.89	82.37	55.36	90.04	75.05	1005	75.66
	ResNet34	80.90	93.99	79.88	73.83	82.00	55.54	90.67	76.13	1103	76.21
	ResNet101	81.00	94.10	79.61	74.36	83.02	56.73	90.47	76.97	1279	76.68
	DLA34	**81.44**	**94.21**	**80.23**	**75.78**	**84.33**	**57.57**	90.82	**78.11**	1189	**76.89**

($y_{a,0} = 1,\ y_{a,1} = 0$), while for unselected lanes, they are assigned as background lanes ($y_{a,0} = 0,\ y_{a,1} = 1$). Finally, the classification loss between all predicted lanes and their labels — assigned based on whether they were selected by the dynamic top-k assigner — is defined as follows:

$$\begin{aligned} \hat{\mathbf{y}} &= \phi(\mathbf{P}_l), \hat{y}_{a,j} \subset \hat{\boldsymbol{y}}(a = 1, 2, ..., N_a, j = 0, 1), \\ \mathcal{L}_{cls} &= -\frac{1}{N_a} \sum_{a=1}^{N_a} \sum_{j=0}^{1} y_{a,j} \cdot \alpha \cdot (1 - \hat{y}_{a,j})^{\gamma} \cdot \log(\hat{y}_{a,j}). \end{aligned} \tag{10}$$

Here, α and γ are hyperparameters used in focal loss [31] to address class imbalance.

Finally, the segmentation loss ($\mathcal{L}_{seg}$) is used as an auxiliary supervision for lane detection. Unlike the previous loss functions that consider only the predicted lanes selected by the dynamic top-k assigner, the segmentation loss utilizes the feature maps extracted by the backbone network within the FPN architecture. In the segmentation decoder in Fig. 4, the three feature maps extracted from the backbone are resized by bilinear interpolation to match the resolution of the largest feature map, with dimensions $\mathbb{R}^{H' \times W' \times C}$ (where $H' = 40,\ W' = 100$). After resizing, the three feature maps are concatenated along the channel dimension, forming a feature map of size ($\mathbb{R}^{H' \times W' \times 3C}$). This concatenated feature map is then passed through a dropout layer and a pointwise convolution, resulting in a feature map of size $\mathbb{R}^{H' \times W' \times N_{cls}}$. Subsequently, this feature map is bilinearly upsampled to match the resolution of the input segmentation labels provided by the dataset. Here, N_{cls} is defined as the maximum number of (GT) lane classes specified for each dataset plus one for the background class. For each pixel $\boldsymbol{p}_j \in \mathbb{R}^{N_{cls}}$ where $j \in HW$, the segmentation loss is calculated using the Negative Log-Likelihood (NLL) loss [32] between the predicted segmentation and the ground truth input segmentation as follows:

$$\begin{aligned} \hat{\boldsymbol{p}}_j &= \log \phi(\boldsymbol{p}_j), \\ \mathcal{L}_{seg} &= -\frac{1}{HW} \sum_{j=1}^{HW} \sum_{c=1}^{N_{cls}} y_{j,c} \log(\hat{p}_{j,c}) \cdot \omega_c. \end{aligned} \tag{11}$$

Here, $\phi(\cdot)$ denotes the softmax function. The index j represents the pixel position along the width and height of the feature map, and c indicates the index within the vector of size N_{cls}. ω_c is the class-specific weight for each lane class, and $y_{j,c}$ is the ground truth label for each pixel, represented using one-hot encoding. The overall loss ($\mathcal{L}$) is formulated as the weighted sum of each individual loss, with corresponding hyperparameters λ, as defined below:

$$\mathcal{L} = \lambda_1 \mathcal{L}_{LaneIoU} + \lambda_2 \mathcal{L}_{reg} + \lambda_3 \mathcal{L}_{seg} + \lambda_4 \mathcal{L}_{cls}. \tag{12}$$

G. Inference

For final lane prediction, we utilize the output from the last stage of the head. During inference, non-maximum suppression (NMS) [16] is applied to predicted lanes whose foreground probability—obtained by applying softmax to the corresponding logit f in $\mathbf{P}_l$—exceeds a predefined threshold

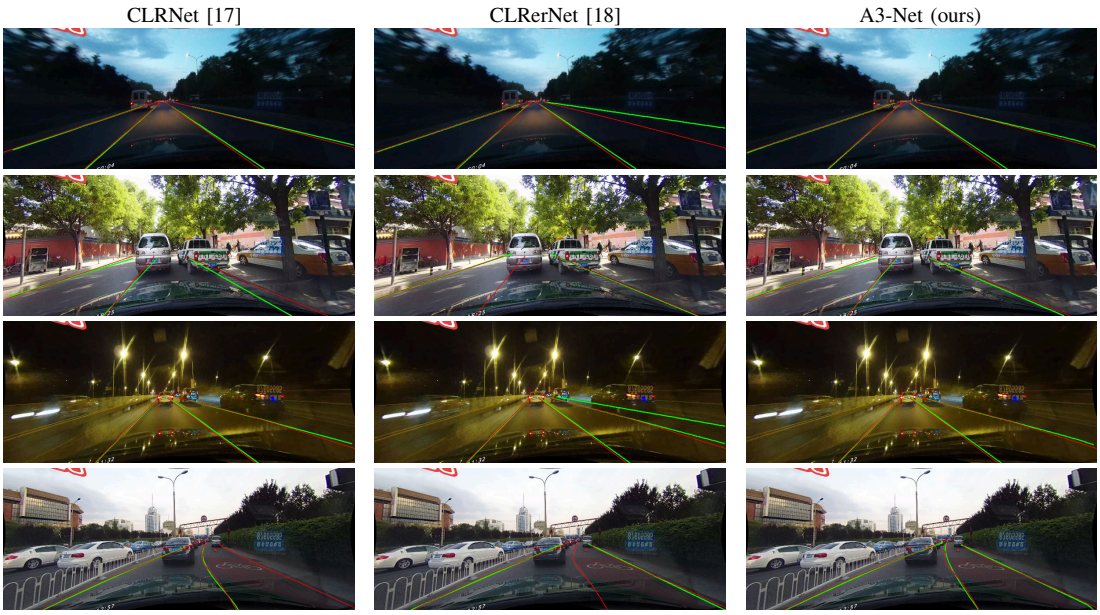


Fig. 6. Visualization results on the CULane [7] dataset comparing anchor-based methods. Column 1: Results of CLRNet, Column 2: Results of CLRerNet, and Column 3: Results of the proposed method. Red line: GT lane, green line: predicted lane.

TABLE II
RESULTS OF THE CURVELANES VALIDATION

Method	F1@50	Precision	Recall
CondLaneNet-S [14]	85.09	87.75	82.58
CondLaneNet-M [14]	85.92	88.29	**83.68**
CondLaneNet-L [14]	86.10	88.98	83.41
CLRNet-DLA34 [17]	86.10	91.40	81.39
CLRerNet-DLA34 [18]	86.47	91.66	81.83
A3-Net (ours)	**86.85**	**91.74**	82.45

TABLE III
SCENARIO PERFORMANCE GROUPED BY LOCAL-GLOBAL CONTEXT

Modules		Both				Local	Global			
AAAM	UEAM	Total	Normal	Shadow	Arrow	Curve	Crowd	Dazzle	Noline	Night
		81.12	94.02	83.71	90.39	74.67	80.20	74.41	56.27	76.53
✓		80.30	**94.23**	84.24	**91.09**	77.98	80.14	74.69	56.14	76.83
	✓	81.17	94.07	84.03	90.81	74.28	80.22	75.33	56.38	76.88
✓	✓	**81.44**	94.21	**84.33**	90.82	**78.11**	**80.23**	**75.78**	**57.57**	**76.89**

τ. Following prior works [16], [17], [18], NMS leverages both the lane presence probability $\mathbf{P}_l$ and the parametric representation $\mathbf{A}_p$ to remove redundant predictions with similar detection regions, thereby effectively limiting the number of detected lanes.

IV. EXPERIMENTAL RESULTS

A. Datasets and Evaluation Metrics

In this paper, we conducted experiments on the lane detection benchmarks CULane [7] and CurveLanes [20]. The CULane dataset is widely used for lane detection, comprising on-road scenarios. It consists of diverse scenarios, including "Normal," "Crowded," "Night," "No line," "Shadow," "Arrow," "Dazzle," "Cross," and "Curve." This dataset includes 88.8K training images, 9.7K validation images, and 34.7K testing images. The image size is 590×1640 pixels. CurveLanes consists of 150K images with challenging curve scenes, most of which have a resolution of 1440×2650 pixels. We selected the F1 score, based on precision and recall values, as the evaluation metric for the CULane and CurveLanes datasets. The true positive (TP) is defined by calculating the IoU between the predicted and GT lanes, classifying any predicted lane with an IoU greater than the threshold (0.5) as a TP.

B. Comparison with State-of-the-art Results

1) Performance on CULane: As shown in Table I, previous lane detection approaches can be grouped into three categories: segmentation-based methods (e.g., SCNN, RESA, UFLD, Lane2Seq (segmentation)) that perform pixel-level lane mask prediction (achieving up to 79.64% F1 score), parameter-based methods (e.g., LSTR, BezierLaneNet, Point2Lane, Lane2Seq (parameter)) that regress explicit curve parameters (up to 79.45% F1 score), and anchor-based methods (e.g., CLRNet, CLRerNet, Lane2Seq (anchor)) that detect lanes via predefined anchors (CLRerNet with DLA-34 attains 81.12% F1 score). In contrast, our proposed approach build on DLA-34 yields a total F1 score of 81.44%, surpassing CLRerNet by +0.32%, and establishes new state-of-the-art performance across all CULane scenarios. The method also delivers consistent improvements on ResNet 18/34/101 backbones (80.66% / 80.90% / 81.00% F1 score). In addition, qualitative results also can be observed in Fig. 6. Our method's lane predictions align more closely with the ground truth in terms of length, angle, and start point—especially for curved lanes—and remain robust even under occlusion or crowded conditions by accurately understanding the underlying road structure.

2) Performance on CurveLanes: As shown in Table II, the proposed method outperformed existing approaches on the CurveLanes dataset, including F1@50 and precision. The proposed model using DLA-34 [28] as the backbone, achieved higher performance across most evaluation metrics compared

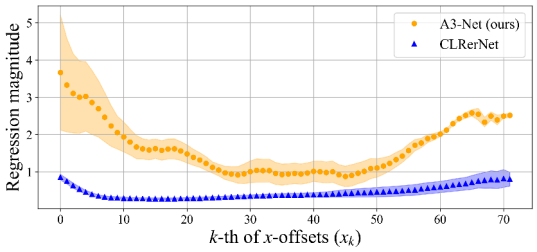


Fig. 7. Total average of the sum of absolute regression values of the x-offsets from each anchor across three heads. Variance is represented as a shaded region around the mean.

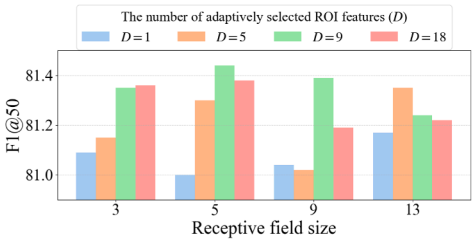


Fig. 8. Analysis of the receptive field and the number of adaptively selected features.

TABLE IV
ABLATION STUDY COMPARING THE PROPOSED AAAM MODULE USING DEPTHWISE ONE-DIMENSIONAL CONVOLUTIONS AGAINST A VARIANT USING STANDARD SQUARE CONVOLUTIONS (3×3)

Categories	DwConv.	Standard Conv.	Enhancement
Total	**81.44**	81.24	**+0.20**
Normal	94.21	**94.23**	-0.02
Crowd	**80.23**	79.92	**+0.31**
Dazzle	**75.78**	75.74	**+0.04**
Shadow	**84.33**	83.89	**+0.44**
Noline	**57.57**	57.09	**+0.48**
Arrow	**90.82**	90.74	**+0.08**
Curve	**78.11**	77.19	**+0.92**
Night	**76.89**	76.53	**+0.36**

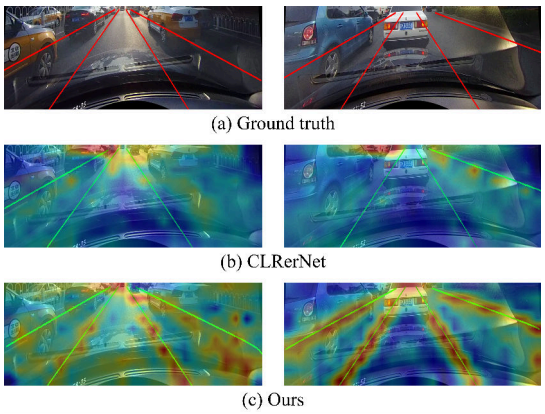


Fig. 9. (a) Image containing the ground truth (GT) lanes (b) global feature map values from CLRerNet (c) embedding-aligned global feature map ($\boldsymbol{z}_{l+1}$) of the proposed method. (red lines: GT, green lines: predicted lanes)

to existing anchor-based models. Additionally, the proposed model surpassed the state-of-the-art model, CLRerNet-DLA-34, by 0.38% in the F1@50 score.

C. Ablation Studies

The CULane dataset categorizes driving scenarios into "Normal", "Crowded", "Night", "No line", "Shadow", "Arrow", "Dazzle", "Cross", and "Curve". Excluding "Normal", which represents general road conditions, and "Cross", where only false positives (FP) are measured to evaluate detection performance [17], [18], the remaining scenarios were considered challenge scenarios. We qualitatively classified these challenge scenarios based on whether precise lane adjustment (local context) or broader scene understanding (global context) is more critical, or whether both aspects are equally important. Specifically, "Curve" was identified as a scenario where local context is essential, as large lane curvatures require fine-grained adjustments to accurately detect lanes. In contrast, "Crowded", "Dazzle", "No line", and "Night" were regarded as global context-dominant scenarios, where lane markings are often occluded, absent, or difficult to perceive due to low illumination—thus necessitating broader contextual reasoning to infer lane positions. Although both local and global information are important across all scenarios, we quantitatively analyzed their relative contributions across different conditions to understand which context plays a more dominant role.

1) Performance Analysis of Each Module on Challenge Scenarios: In this study, we evaluated the F1@50 score under four different conditions for the following challenge scenarios: "Curve," where local detail was considered important; "Crowded", "Dazzle", "No line", and "Night", where global context was regarded as critical; and "Shadow" and "Arrow", where both local and global information needed to be jointly considered. The results of this ablation study are presented in Table III: (a) using the baseline model CLRerNet [15]; (b) employing the AAAM, which captured low-level information and excelled in local context scenarios; (c) utilizing the UEAM, which leveraged high-level information for global context scenarios; and (d) combining both AAAM and UEAM to harness the advantages of both local and global contexts, thereby achieving superior performance across all challenging scenarios. As indicated in Table III, in scenarios where local context was critical, the AAAM significantly improved lane detection performance by adaptively extracting key information from the ROI features. Conversely, in scenarios where global context was important, the UEAM, which aligned and integrated the embeddings of the ROI features and the global feature map, substantially enhanced detection performance. By utilizing both modules together, we effectively leveraged features that incorporating both local and global information, achieving state-of-the-art performance.

2) Analysis of the x-Offset Regression: As mentioned in Section II-E, the final prediction of the proposed model includes the length, angle, starting point, and x-offset from a straight anchor. The accuracy of lane prediction is primarily determined by the x-offset, making regression crucial. The proposed model achieved superior performance in the curve category on the CULane dataset [7] compared to the existing methods [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19]. To analyze this, we examined the changes in the x-offsets, as illustrated in Fig. 7, which shows the average sum of the absolute values of the x-offsets from each anchor for images in the curve category of the CULane dataset. This

TABLE V
DETAILED ARCHITECTURE OF A3-NET, SHOWING THE BACKBONE, FPN, HEAD, AND SEGMENTATION. DETAILED STRUCTURE OF THE BASIC BLOCK IS SHOWN IN TABLE VI

Stage	Output Size	Module	#Blocks	In → Out Channel	Kernel/Stride
Backbone (DLA-34)	H×W	Conv7×7 + BN + ReLU	–	3→16	7×7 / 1
	H×W	Conv3×3 + BN + ReLU	–	16→16	3×3 / 1
	H/2×W/2	Conv3×3 + BN + ReLU	–	16→32	3×3 / 2
	H/4×W/4	BasicBlock ×2	2	32→64	Conv1:3×3/2, Conv2:3×3/1
	H/8×W/8	Level3: BasicBlock ×4	4	64→128	Conv1:3×3/2, Conv2:3×3/1
	H/16×W/16	Level2: BasicBlock ×4	4	128→256	Conv1:3×3/2, Conv2:3×3/1
	H/32×W/32	Level1: BasicBlock ×2	2	256→512	Conv1:3×3/2, Conv2:3×3/1
FPN	H/8×W/8	1×1 Conv on Backbone Level3 (128→64)	–	128→64	1×1 / 1
		Upsample from FPN Level2 → Level3 Resolution	–	–	×2 / Nearest
		3×3 Conv on Merged Level3 Feature (64→64)	–	64→64	3×3 / 1
	H/16×W/16	1×1 Conv on Backbone Level2 (256→64)	–	256→64	1×1 / 1
		Upsample from FPN Level1 → Level2 Resolution	–	–	×2 / Nearest
		3×3 Conv on Merged Level2 Feature (64→64)	–	64→64	3×3 / 1
	H/32×W/32	1×1 Conv on Backbone Level1 (512→64)	–	512→64	1×1 / 1
		3×3 Conv on Merged Level1 Feature (64→64)	–	64→64	3×3 / 1
Head	–	AnchorGenerator (192)	–	–	–
	–	AAAM: DwConv(1×3, groups=64) ×2 + Linear(72→9)	–	–	–
	–	UEAM ×9	9	64→64	–
	–	ConvModule (9×1, padding=4) ×3	3	64→48→64	9×1 / 1
	–	ConcatConv (9×1, padding=4) ×3	3	48→144→64	9×1 / 1
	–	MLP: 442→352→262→192	–	576→576	–
	–	Cls. Head: (576→576→ReLU) x2→2	–	576→2	–
	–	Reg. Head: (576→576→ReLU) x2→76	–	576→76	–
Segmentation	–	Segmentation Decoder: Dropout + Conv(192→5)	–	192→5	–

TABLE VI
DETAILED STRUCTURE OF THE BASIC BLOCK

Layer	Kernel/Stride	In → Out Channel	Activation
Conv1	3×3 / 2	$C_{\text{in}} \rightarrow C_{\text{out}}$	ReLU
BatchNorm1	–	–	–
Conv2	3×3 / 1	$C_{\text{out}} \rightarrow C_{\text{out}}$	–
BatchNorm2	–	–	–

TABLE VII
DETAILED HYPERPARAMETER SETTINGS FOR CULANE AND CURVELANES

Dataset	$\min k$ / ω_3	$\max k$ / ω_4	w_{lane} / N_{cls}	α / τ	γ / λ_1	ω_0 / λ_2	ω_1 / λ_3	ω_2 / λ_4
CULane	1	4	15/800	0.25	2.0	0.4	1.0	1.0
	1.0	1.0	5	0.41	4.0	0.2	1.0	2.0
CurveLanes	1	4	5/224	0.25	2.0	0.4	1.0	–
	–	–	2	0.43	4.0	0.2	2.0	2.0

result demonstrates that the proposed model can regress lanes with a high degree of freedom, as evidenced by a greater variance in the x-offset changes compared to CLRerNet [18].

3) Analysis of Convolution Operation in AAAM: As mentioned in Section III-C, AAAM applied depth-wise one-dimensional convolution on a per-channel basis to prevent information mixing between anchors and to more precisely capture relevant features. Table IV compares the performance when using a standard convolution kernel (3×3) versus the proposed depth-wise one-dimensional convolution during ROI feature processing in AAAM, and demonstrates the performance improvement achieved by our approach over standard convolution. As shown in Table IV, when using a standard convolution, there was a risk of feature mixing between different anchors, leading to the loss of anchor-specific information and a degradation in overall performance. In contrast, applying depth-wise one-dimensional convolution—which prevented information from mixing across anchors—resulted in improved overall performance, with particularly significant gains observed in the "Curve" scenario.

4) Ablation Study on Anchor Receptive Field and the Number of Adaptively Selected ROI Features in AAAM: We conducted an ablation study to analyze the effect of anchor point receptive field size and the number of adaptively selected ROI features ($\widehat{\boldsymbol{\mathcal{X}}} \in \mathbb{R}^{N_a \times C \times D}$) for each ROI feature ($\boldsymbol{\mathcal{X}} \in \mathbb{R}^{N_a \times C \times S}$) within the AAAM module, where N_a is the number of anchors, C is the channel size, D and S are the number of adaptively selected ROI features ($\widehat{\boldsymbol{\mathcal{X}}}$) and original ROI features ($\boldsymbol{\mathcal{X}}$), respectively. Specifically, we varied the receptive field size, which determines the region referenced by points within an anchor. This was achieved by adjusting the number of convolutional layers and the kernel size used in the AAAM module, and the number (D) of adaptively selected ROI features ($\widehat{\boldsymbol{\mathcal{X}}}$). As illustrated in Fig. 8, increasing the receptive field size from three to five generally led to notable improvements in F1@50 across most settings, with the best performance achieved when the receptive field size was set to five and the number of adaptively selected ROI features ($\widehat{\boldsymbol{\mathcal{X}}}$) was set to nine. This setting allowed the model to capture richer local context while avoiding excessive smoothing that could occur with overly large receptive fields. Additionally, we observed that increasing the number (D) of adaptively selected ROI feature ($\widehat{\boldsymbol{\mathcal{X}}}$) from one to nine generally improved the performance, indicating that incorporating multiple adaptively selected features enhanced the model's ability to represent complex lane patterns. However, setting the number (D) of adaptively selected ROI features ($\widehat{\boldsymbol{\mathcal{X}}}$) too high (e.g., 18) resulted in a slight drop, likely due to feature redundancy and overfitting to local noise. Overall, these results highlighted the importance of carefully balancing the receptive field size and the number (D) of adaptively selected features ($\widehat{\boldsymbol{\mathcal{X}}}$) to maximize the effectiveness of the AAAM.

5) Analysis of the embedding alignment in UEAM: As shown in Fig. 9(c), we visualized the output $\boldsymbol{z}_{l+1}$ from the global pathway and compared it with the global feature map of CLRerNet in Fig. 9(b). In CLRerNet, since the global feature map was trained to reflect the overall contextual information of the image, high response values were distributed even in regions unrelated to the lanes. In contrast, our method performed embedding alignment between the sliced ROI feature ($\widehat{\boldsymbol{x}}_l$) and the global feature map ($\boldsymbol{z}'_l$), resulting in a global feature map that exhibits stronger responses concentrated in the regions corresponding to the lanes. Consequently, when $\widehat{\boldsymbol{x}}_l$ was used as the query and $\boldsymbol{z}_{l+1}$ was used as the key and value during the multi-head attention (MHA) process, higher attention scores were assigned in the regions corresponding to the lanes. Theses results demonstrated that the embedding alignment process of UEAM effectively integrated local contextual information and global contextual information, thereby improving the accuracy of lane detection even under challenging scenarios where lanes were occluded.

D. Implementation Details

We employed ResNet [22] or DLA [28] as the backbone network. Input images were resized to 320×800 pixels, and standard data augmentation techniques were applied, including random affine transformations (translation, rotation, scaling) and random horizontal flipping. The architectural hidden dimensions are summarized in Table V, and the input/output channel configurations of each basic block are detailed in Table VI. A learning rate warm-up strategy [33] was adopted, where the learning rate was initially set to one-tenth of the maximum and gradually increased until stabilization. After reaching the peak learning rate of 6×10^{-4}, a cosine annealing schedule was applied to decay it smoothly to zero. Training was conducted for 15 epochs on the CULane [7] dataset and 30 epochs on the CurveLanes [20] dataset, following [18]. The proposed model was implemented in PyTorch and trained on a single NVIDIA RTX 4090 GPU. The total batch size was set to 24; however, when using ResNet-101 [22] as the backbone, the batch size was reduced to 12, and the learning rate to 3×10^{-4}. During inference, non-maximum suppression (NMS), following the method in [16], was used during inference to eliminate highly overlapping predictions. Detailed training and inference hyperparameters are provided in Table VII. The hyperparameters $\min k$ and $\max k$ configure the dynamic top-k assigner; w_{lane}, α, γ, ω_k, N_{cls}, and λ are defined in Equations (6), (10), (11), and (12), respectively; and τ denotes the threshold in Section III-G.

V. Conclusion

In this paper, we addressed the limitations of anchor-based lane detection methods under challenging real-world conditions by proposing the Adaptive Anchor Aggregation and Unified Embedding Alignment Network (A3-Net)—an end-to-end architecture comprising two key modules: the Adaptive Anchor Aggregation Module (AAAM) and the Unified Embedding Alignment Module (UEAM). The AAAM dynamically selected region-of-interest (ROI) features according to lane shape, while the UEAM aligned local and global contextual embeddings within a shared feature space. This alignment between the global feature map and ROI feature embeddings enabled the subsequent cross-attention process to compute higher similarity scores in lane regions between ROI feature and global feature map for lane regions compared to previous methods, demonstrating that through UEAM, global features effectively reflected the local characteristics of ROI features. On the CULane benchmark dataset—one of the most widely used and representative benchmarks for lane detection—A3-Net achieved an overall F1 score of 81.44% (+0.32%). It effectively fused ROI and global feature maps, yielding notable gains in scenarios demanding local contextual information (curved: +2.54%), global contextual information (crowded: +0.03%, dazzle: +0.48%, no line: +1.3%, night: +0.36%), and both (normal: +0.19%, shadow: +0.62%), thereby outperforming state-of-the-art approaches. These results validated the effectiveness of adaptive feature aggregation and embedding alignment for robust lane detection. In future work, we plan to evaluate A3-Net under adverse weather conditions such as rain and snow, and to explore its integration with multi-modal sensors (e.g., radar, LiDAR) to further enhance detection performance in safety-critical applications.

References

[1] J. Tang, S. Li, and P. Liu, "A review of lane detection methods based on deep learning," *Pattern Recognit.*, vol. 111, p. 107623, 2021.

[2] O. M. S. Ali, A. D. Al-Zuky, and F. E. M. Al-Obaidi, "R-CNN Auto-system for detecting text road signs in Baghdad," *IEIE Trans. Smart Process. Comput.*, vol. 13, no. 2, pp. 140–147, 2024.

[3] J. Choi, J. Lee, K. Kim, and S. Kim, "Efficient object detection acceleration methods for autonomous-driving embedded platforms," *IEIE Trans. Smart Process. Comput.*, vol. 11, no. 4, pp. 255–261, Aug. 2022.

[4] J. Choi, D. Chun, H.-J. Lee, and H. Kim, "Efficient object detection acceleration methods for autonomous-driving embedded platforms," *IEIE Trans. Smart Process. Comput.*, vol. 11, no. 4, pp. 255–261, 2022.

[5] B.-U. Jeon and K. Chung, "Dynamic framerate SlowFast network for improving autonomous driving performance," *IEIE Trans. Smart Process. Comput.*, vol. 12, no. 3, pp. 261–268, 2023.

[6] S. Lee et al., "VPGNet: Vanishing point guided network for lane and road marking detection and recognition," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 1965–1973.

[7] X. Pan, J. Shi, P. Luo, X. Wang, and X. Tang, "Spatial as deep: Spatial CNN for traffic scene understanding," in *Proc. 32nd AAAI Conf. Artif. Intell.*, 2018, pp. 1–8.

[8] T. Zheng et al., "RESA: Recurrent feature-shift aggregator for lane detection," in *Proc. 35th AAAI Conf. Artif. Intell.*, 2021, pp. 3547–3554.

[9] R. Liu, Z. Yuan, T. Liu, and Z. Xiong, "End-to-end lane shape prediction with transformers," in *Proc. IEEE Winter Conf. Appl. Comput. Vis. (WACV)*, Jan. 2021, pp. 3693–3701.

[10] L. Tabelini, R. Berriel, T. M. Paixão, C. Badue, A. F. De Souza, and T. Oliveira-Santos, "PolyLaneNet: Lane estimation via deep polynomial regression," in *Proc. 25th Int. Conf. Pattern Recognit. (ICPR)*, Jan. 2021, pp. 6150–6156.

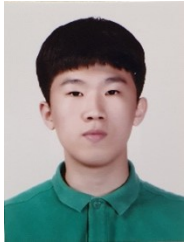
[11] Z. Feng, S. Guo, X. Tan, K. Xu, M. Wang, and L. Ma, "Rethinking efficient lane detection via curve modeling," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 17041–17049.

[12] Y. J. Chae, S. J. Park, E. S. Kang, M. J. Chae, B. H. Ngo, and S. I. Cho, "Point2Lane: Polyline-Based Reconstruction with Principal Points for Lane Detection," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 12, pp.14813–14829, Dec. 2023, doi: 10.1109/TITS.2023.3295807

[13] Z. Qin, P. Zhang, and X. Li, "Ultra fast deep lane detection with hybrid anchor driven ordinal classification," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 5, pp. 2555–2568, May 2024, doi: 10.1109/TPAMI.2022.3182097.

[14] L. Liu, X. Chen, S. Zhu, and P. Tan, "CondLaneNet: A top-to-down lane detection framework based on conditional convolution," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 3753–3762.

[15] X. Li, J. Li, X. Hu, and J. Yang, "Line-CNN: End-to-end traffic line detection with line proposal unit," *IEEE Trans. Intell. Transp. Syst.*, vol. 21, no. 1, pp. 248–258, Jan. 2020, doi: 10.1109/TITS.2019.2890870.
[16] L. Tabelini, R. Berriel, T. M. Paixão, C. Badue, A. F. De Souza, and T. Oliveira-Santos, "Keep your eyes on the lane: Real-time attention-guided lane detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 294–302.
[17] T. Zheng et al., "CLRNet: Cross layer refinement network for lane detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 888–897.
[18] H. Honda and Y. Uchida, "CLRerNet: Improving confidence of lane detection with LaneIoU," in *Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis.*, Jan. 2024, pp. 1176–1185.
[19] K. Zhou, "Lane2Seq: Towards unified lane detection via sequence generation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024, pp. 16944–16953.
[20] H. Xu, S. Wang, X. Cai, W. Zhang, X. Liang, and Z. Li, "CurveLaneNAS: Unifying lane-sensitive architecture search and adaptive point blending," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, Nov. 2020, pp. 689–704.
[21] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
[22] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.
[23] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, and others, "ImageNet large scale visual recognition challenge," *Int. J. Comput. Vis.*, vol. 115, pp. 211–252, 2015.
[24] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature pyramid networks for object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 936–944.
[25] H. Rezatofighi, N. Tsoi, J. Gwak, A. Sadeghian, I. Reid, and S. Savarese, "Generalized intersection over union: A metric and a loss for bounding box regression," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2019, pp. 658–666.
[26] T. Yao, Y. Li, Y. Pan, Y. Wang, X.-P. Zhang, and T. Mei, "Dual vision transformer," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 9, pp. 10870–10882, Sep. 2023, doi: 10.1109/TPAMI.2023.3268446.
[27] V. Nair and G. E. Hinton, "Rectified linear units improve restricted Boltzmann machines," in *Proc. 27th Int. Conf. Mach. Learn. (ICML)*, 2010, pp. 807–814.
[28] F. Yu, D. Wang, E. Shelhamer, and T. Darrell, "Deep layer aggregation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2018, pp. 2403–2412.
[29] F. Chollet, "Xception: Deep learning with depthwise separable convolutions," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2017, pp. 1251–1258.
[30] P. J. Huber, "Robust estimation of a location parameter," in *Breakthroughs in Statistics: Methodology and Distribution*, Springer, 1992, pp. 492–518.
[31] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 2980–2988.
[32] R. A. Fisher, "On the mathematical foundations of theoretical statistics," *Philosophical Transactions of the Royal Society of London. Series A, Containing Papers of a Mathematical or Physical Character*, vol. 222, pp. 309–368, 1922.
[33] P. Goyal et al., "Accurate, large minibatch SGD: Training ImageNet in 1 hour," 2017. [Online]. Available: https://doi.org/10.48550/arXiv.1706.02677. arXiv:1706.02677.



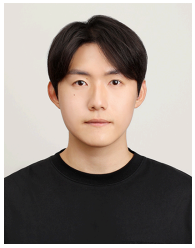
Jae Yun Lee received the B.S. degree in Multimedia Engineering from Dongguk University, Seoul Republic of Korea, in 2024, where he is currently pursuing the M.S. degree with the Department of AI Software Convergence. His current research interests include domain adaptation and lane detection..



Seon Ho Lee received the B.S. degree in Multimedia Engineering from Dongguk University, Seoul, Republic of Korea, in 2025, where he is currently pursuing the M.S. degree with the Department of AI Software Convergence. His research interests include hand gesture recognition, lane detection, and computer graphics.



Sang Hoon Han received the B.S. degree in Multimedia Engineering from Dongguk University, Seoul, Republic of Korea, in 2024, where he is currently pursuing the M.S. degree with the Department of AI Software Convergence. His research interests include lane detection, hand gesture recognition, and EEG signal processing.



Geunha Kim is currently pursuing a M.S. degree at Hanyang University, Seoul, Republic of Korea. His current research interests include lane detection, object centric learning and meta reinforcement learning.



Jae Hyeon Park received a B.S. in Electronic Engineering from Daegu University, Republic of Korea, in 2019 and an M.S. in Multimedia Engineering from Dongguk University, Seoul, Republic of Korea, in 2021. He earned a Ph.D. in Computer Science from Dongguk University, Seoul, Korea. His research interests include semi/self-supervised learning, domain adaptive object detection, and image generation. He is currently a Postdoctoral Research Fellow with the Division of AI Software Convergence, at Dongguk University.



Gyoomin Lee received the B.S. degree in Multimedia Engineering from Dongguk University, Seoul, Republic of Korea, in 2024. He is currently pursuing a M.S. degree at Dongguk University, Seoul, Republic of Korea. His current research interests include semi/self-supervised learning, few-shot learning and unsupervised content-based image retrieval.



Sung In Cho (Member, IEEE) received the B.S.degree in electronic engineering from Sogang University, Seoul, Republic of Korea, in 2010, and he received the Ph.D. degree in electrical and computer engineering from the Pohang University of Science and Technology, Pohang, Republic of Korea in 2015. From 2015 to 2017, he was a Senior Researcher with LG Display. From 2017 to 2019, he was an Assistant Professor of Electronic Engineering at Daegu University, Gyeongsan, Republic of Korea. He is currently an Associate Professor of Department of AISW convergence at Dongguk University, Seoul. His current research interests include image analysis and enhancement, video processing, computer vision, and deep learning.